

Tensorflow II

Classical Image Processing

Fatemeh Dehrouyeh



FANSHAWE

Contents

- 1 Classical Image Processing
 - Why Classical Image Processing?
 - Brightness and Contrast
 - Noise
 - Filtering and Blurring
 - Convolution and Kernels
 - Gaussian Blur
 - Sharpening
 - Edges and Gradients
 - Thresholding and Masks
 - Morphological Operations
 - Connected Components and Object Counting
 - Histograms and Contrast
 - Color Spaces
 - Geometric Transformations

Current Section

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Why Study Classical Image Processing?

Before deep learning, computer vision relied heavily on manually designed image operations.

These operations are still useful because they help us understand:

- how images change numerically,
- how noise affects images,
- how filters work,
- how edges are detected,
- why convolution is important.

Connection to CNNs

Many classical image-processing operations are based on local neighborhoods, which is the same core idea behind convolutional neural networks.

Classical Processing vs. Deep Learning

Aspect	Classical Vision	Deep Learning
Features	Hand-designed	Learned from data
Examples	Edges, corners, textures	CNN filters, embeddings
Rules	Written by humans	Optimized during training
Data need	Usually lower	Usually higher
Flexibility	Limited	High

Key Idea

Classical methods help us understand what CNNs learn automatically.

Image Operations as Transformations

An image-processing operation transforms an input image into an output image:

Input Image $\longrightarrow$ Processing Operation $\longrightarrow$ Output Image

Examples:

- brightness adjustment,
- contrast adjustment,
- blurring,
- sharpening,
- edge detection,
- thresholding.

TensorFlow View

In TensorFlow, many image operations are tensor transformations.

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- **Brightness and Contrast**
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Brightness Adjustment

Brightness controls how light or dark an image appears.

A simple brightness adjustment adds a constant value to every pixel:

$$I_{\text{new}} = I + b$$

where b controls the brightness change.

- $b > 0$: image becomes brighter
- $b < 0$: image becomes darker

Important

Pixel values should remain in a valid range, such as $[0, 1]$ or $[0, 255]$.

Sample Image

```
import tensorflow as tf
import matplotlib.pyplot as plt
from skimage import data

# Load coffee image
img = data.coffee()

# Convert NumPy array to TensorFlow tensor
img = tf.convert_to_tensor(img)

print(img.shape)

# Display
plt.imshow(img)
plt.axis("off")
plt.show()
```

... (400, 600, 3)



Code

```
import tensorflow as tf
import matplotlib.pyplot as plt

img_float = tf.cast(img, tf.float32) / 255.0

brighter = tf.image.adjust_brightness(img_float, delta=0.25)
darker = tf.image.adjust_brightness(img_float, delta=-0.25)

brighter = tf.clip_by_value(brighter, 0.0, 1.0)
darker = tf.clip_by_value(darker, 0.0, 1.0)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

axes[0].imshow(img_float)
axes[0].set_title("Original")

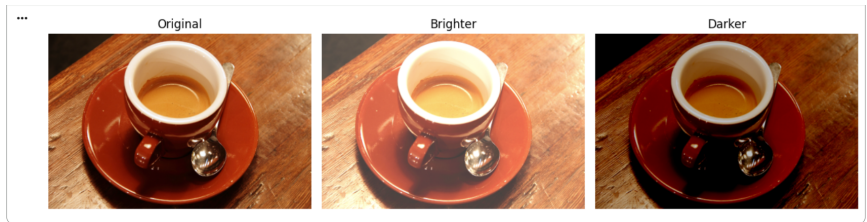
axes[1].imshow(brighter)
axes[1].set_title("Brighter")

axes[2].imshow(darker)
axes[2].set_title("Darker")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Code—Result



Contrast Adjustment

Contrast controls the difference between dark and bright regions.

A high-contrast image has stronger differences between pixel values.

A low-contrast image has pixel values that are closer together.

TensorFlow provides:

```
tf.image.adjust_contrast()
```

Intuition

Brightness shifts pixel values, while contrast spreads or compresses them.

Code

```
▶ low_contrast = tf.image.adjust_contrast(img_float, contrast_factor=0.5)
  high_contrast = tf.image.adjust_contrast(img_float, contrast_factor=1.8)

  low_contrast = tf.clip_by_value(low_contrast, 0.0, 1.0)
  high_contrast = tf.clip_by_value(high_contrast, 0.0, 1.0)

  fig, axes = plt.subplots(1, 3, figsize=(12, 4))

  axes[0].imshow(img_float)
  axes[0].set_title("Original")

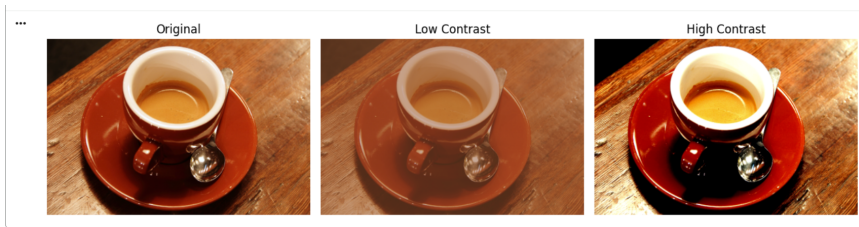
  axes[1].imshow(low_contrast)
  axes[1].set_title("Low Contrast")

  axes[2].imshow(high_contrast)
  axes[2].set_title("High Contrast")

  for ax in axes:
      ax.axis("off")

  plt.tight_layout()
  plt.show()
```

Code—Result



Contrast adjustment works by measuring how far each pixel is from the mean brightness of the image, multiplying that difference by the contrast factor, and then adding the mean back: $\text{new pixel} = (\text{old pixel} - \text{mean}) \times \text{factor} + \text{mean}$. A factor less than 1 moves pixel values closer to the mean and reduces contrast, while a factor greater than 1 pushes pixel values farther from the mean and increases contrast.

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- **Noise**
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

What Is Image Noise?

Noise is unwanted random variation in pixel values.

Noise can come from:

- camera sensors,
- low-light conditions,
- compression,
- transmission errors,
- data corruption.

Why It Matters

Noise can hide important visual patterns and make detection, classification, and segmentation harder.

Code

```
▶ noise = tf.random.normal(shape=tf.shape(img_float), mean=0.0, stddev=0.08)

noisy = img_float + noise
noisy = tf.clip_by_value(noisy, 0.0, 1.0)

fig, axes = plt.subplots(1, 2, figsize=(9, 4))

axes[0].imshow(img_float)
axes[0].set_title("Original")

axes[1].imshow(noisy)
axes[1].set_title("Noisy Image")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Code—Result



Gaussian noise is added by generating a random value for every pixel using a normal distribution with mean 0 and standard deviation 0.08. The noise has the same shape as the image, so each pixel and color channel receives its own random value. This noise is then added to the original image: $\text{noisy pixel} = \text{original pixel} + \text{random noise}$. Because some values may become less than 0 or greater than 1, the result is clipped to the valid range $[0, 1]$.

Noise and Pixel Values

Adding noise changes pixel values randomly.

For example, a pixel value may change from

$$0.62 \longrightarrow 0.70$$

or

$$0.62 \longrightarrow 0.51$$

The visual content is still similar, but the numerical image has changed.

Important

Deep learning models should be robust to small changes in pixel values.

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- **Filtering and Blurring**
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Why Do We Blur Images?

Blurring smooths an image by reducing small local variations in pixel values.

It is commonly used to:

- reduce noise,
- remove small details,
- smooth textures,
- prepare an image before edge detection.

Key Idea

Blurring replaces each pixel using information from nearby pixels.

Code

```
▶ # TensorFlow does not have a direct Gaussian blur function in basic tf.image,  
# so we can use average pooling as a simple blur.  
  
img_batch = tf.expand_dims(img_float, axis=0)  
  
blurred = tf.nn.avg_pool2d(  
    img_batch,  
    ksize=5,  
    strides=1,  
    padding="SAME"  
)  
  
blurred = tf.squeeze(blurred, axis=0)  
  
fig, axes = plt.subplots(1, 2, figsize=(9, 4))  
  
axes[0].imshow(img_float)  
axes[0].set_title("Original")  
  
axes[1].imshow(blurred)  
axes[1].set_title("Average Blur")  
  
for ax in axes:  
    ax.axis("off")  
  
plt.tight_layout()  
plt.show()
```

Code—Result

...

Original



Average Blur



Average pooling can be used as a simple blur by replacing each pixel with the average value of its surrounding neighborhood. With `kernel_size=5`, TensorFlow uses a 5×5 region around each pixel, so the new pixel value is approximately $\text{new pixel} = \frac{\text{sum of the 25 neighboring pixel values}}{25}$. This averaging is performed separately for the red, green, and blue channels. A stride of 1 moves the averaging window one pixel at a time, while `padding="SAME"` keeps the output image the same size as the input. The image is temporarily given a batch dimension because `avg_pool2d` expects batched input, and this dimension is removed afterward.

Average Filtering

An average filter replaces each pixel with the average of its neighbors.

For example, using a 3×3 neighborhood:

$$I_{\text{new}}(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(x + i, y + j)$$

This reduces sharp changes in the image.

Trade-off

Average filtering can reduce noise, but it also removes details and makes edges less sharp.

Code

```
▶ blur_3 = tf.nn.avg_pool2d(img_batch, ksize=3, strides=1, padding="SAME")
  blur_9 = tf.nn.avg_pool2d(img_batch, ksize=9, strides=1, padding="SAME")
  blur_15 = tf.nn.avg_pool2d(img_batch, ksize=15, strides=1, padding="SAME")

  blur_3 = tf.squeeze(blur_3, axis=0)
  blur_9 = tf.squeeze(blur_9, axis=0)
  blur_15 = tf.squeeze(blur_15, axis=0)

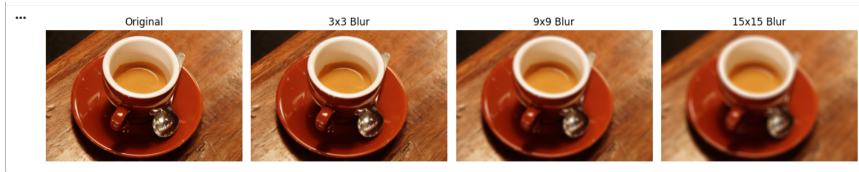
  fig, axes = plt.subplots(1, 4, figsize=(14, 4))

  images = [
      (img_float, "Original"),
      (blur_3, "3x3 Blur"),
      (blur_9, "9x9 Blur"),
      (blur_15, "15x15 Blur")
  ]

  for ax, (im, title) in zip(axes, images):
      ax.imshow(im)
      ax.set_title(title)
      ax.axis("off")

  plt.tight_layout()
  plt.show()
```

Code—Result



Filtering as a Local Operation

Filtering means computing each output pixel from a small neighborhood around the corresponding input pixel.

New pixel = function of nearby pixels

Examples:

- average of nearby pixels,
- weighted average of nearby pixels,
- difference between neighboring pixels,
- maximum or minimum in a neighborhood.

Connection to CNNs

A convolutional layer also looks at local neighborhoods, but its filter weights are learned from data.

Main Image Smoothing Filters Compared

Filter	How it works	Good for	Edge preservation
Average	Equal-weight average	Simple smoothing	Poor
Gaussian	Weighted average; center pixels count more	Gaussian-like noise, general smoothing	Moderate/Poor
Median	Replaces pixel with neighborhood median	Salt-and-pepper noise	Good
Bilateral	Uses spatial + intensity similarity	Noise reduction while keeping edges	Very good

Simple Memory Rule

Average = equal weights Gaussian = center matters more Median = removes outliers
 Bilateral = preserves edges

Note: Salt-and-pepper noise is a type of image noise where random pixels are replaced by very dark or very bright values.

Main Image Smoothing Filters

Average Filter: The average filter replaces each pixel with the arithmetic mean of the surrounding pixels in a local neighborhood. Every neighboring pixel receives the same weight, so the filter reduces small intensity variations and smooths the image, but it also tends to blur important edges because pixels from both sides of an edge are averaged together.

Gaussian Filter: The Gaussian filter averages neighboring pixels using larger weights for pixels closer to the center and smaller weights for pixels farther away.

It computes new pixel = $\frac{\sum(\text{neighbor pixel} \times \text{Gaussian weight})}{\sum \text{Gaussian weights}}$, so nearby pixels contribute more strongly to the new pixel value. This produces smoother and more natural blurring than a simple average filter, but strong edges can still become blurred because the filter does not consider whether neighboring pixels have similar intensities.

Median Filter: The median filter collects the pixel values inside a local neighborhood, sorts them, and replaces the center pixel with the median value instead of the average. Because extreme values do not strongly affect the median, this filter is especially effective for removing salt-and-pepper noise while preserving edges better than average or Gaussian filtering.

Bilateral Filter: The bilateral filter calculates a weighted average using both spatial distance and intensity similarity, where weight = spatial weight $\times$ intensity weight.

It then computes new pixel = $\frac{\sum(\text{neighbor pixel} \times \text{weight})}{\sum \text{weights}}$, so nearby pixels with similar intensity or color contribute most strongly to the new pixel value. Pixels with very different intensity receive very small weights even if they are physically close, which reduces noise while preserving sharp edges.

Padding Modes: VALID vs SAME

VALID Padding: No padding is added around the image. The filter is only applied where the kernel completely fits inside the image, so the output becomes smaller.

SAME Padding: Enough padding is added around the image so that, with stride 1, the output has the same height and width as the input. In TensorFlow convolution and pooling operations, `padding="SAME"` controls the output size rather than describing the exact border values.

Simple Memory Rule

VALID = no padding, output shrinks SAME = padding added, output size stays the same

Border Padding Methods

Zero Padding: Adds zeros around the image border. It is simple and commonly used, but it can introduce artificial dark values near the edges.

Reflect Padding: Extends the image by mirroring pixel values at the border. This usually produces smoother and more natural behavior near image boundaries.

Replicate Padding: Extends the image by repeating the nearest edge pixel outward. This preserves the border intensity and avoids introducing artificial zeros.

Simple Memory Rule

ZERO = add 0s

REFLECT = mirror border
pixels

REPLICATE = repeat edge

Border Padding Methods: Example

Suppose the original row is:

1 2 3 4

Zero Padding: Adds zeros around the border.

0 0 | 1 2 3 4 | 0 0

Reflect Padding: Mirrors pixel values at the border.

3 2 | 1 2 3 4 | 3 2

Replicate Padding: Repeats the nearest edge pixels outward.

1 1 | 1 2 3 4 | 4 4

Simple Memory Rule

ZERO = add 0s

REFLECT = mirror border
pixels

REPLICATE = repeat edge

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- **Convolution and Kernels**
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

What Is a Kernel?

A **kernel** (or filter) is a small matrix that moves across an image and computes a new value from each local neighborhood.

For example, a 3×3 kernel:

$$K = \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{bmatrix}$$

is applied repeatedly across the image.

Key Idea

Different kernel values produce different effects such as **blurring, sharpening, or edge detection**.

Note: Stride specifies how many pixels the filter moves horizontally and vertically each time it advances across the image. The filter moves horizontally across the image until it reaches the end of a row, using the horizontal stride as its step size, then moves down by the vertical stride and continues scanning the next row.

Code—applying a custom kernel in TensorFlow

```
import tensorflow as tf
import matplotlib.pyplot as plt

# img_float should have shape (H, W, 3)
# and values in [0, 1]

img_batch = tf.expand_dims(img_float, axis=0)

# 3x3 average kernel
kernel = tf.ones((3, 3, 3, 1), dtype=tf.float32) / 9.0

# Apply the average filter independently to each RGB channel
blurred = tf.nn.depthwise_conv2d(
    img_batch,
    filter=kernel,
    strides=[1, 1, 1, 1],
    padding="SAME"
)

blurred = tf.squeeze(blurred, axis=0)

# Compare original and blurred images
fig, axes = plt.subplots(1, 2, figsize=(9, 4))

images = [
    (img_float, "Original"),
    (blurred, "3x3 Average Filter")
]

for ax, (im, title) in zip(axes, images):
    ax.imshow(tf.clip_by_value(im, 0, 1))
    ax.set_title(title)
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Code—Result

...

Original



3x3 Average Filter



How Does Convolution Work?

At each image location:

- 1 Place the kernel over a small image region.
- 2 Multiply corresponding values.
- 3 Add the results.
- 4 Store the result as a new output pixel.
- 5 Move the kernel to the next location.

For one location:

$$\text{Output} = \sum_i \sum_j I(i,j) K(i,j)$$

Why This Matters

This operation is the foundation of both **classical image filtering** and **CNNs**.

A Simple Convolution Example

Suppose an image neighborhood is

$$I = \begin{bmatrix} 10 & 20 & 30 \\ 20 & 30 & 40 \\ 30 & 40 & 50 \end{bmatrix}$$

and we use an averaging kernel:

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Then the output value is:

$$\frac{10 + 20 + 30 + 20 + 30 + 40 + 30 + 40 + 50}{9} = 30$$

Interpretation

The center pixel is replaced by the average intensity of its neighborhood.

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- **Gaussian Blur**
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Gaussian Blur

A Gaussian filter also smooths an image, but nearby pixels are **not weighted equally**.

Pixels closer to the center receive larger weights.

A simple Gaussian-like kernel is:

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Compared with Average Blur

Gaussian blur usually produces smoother and more natural-looking results because nearby pixels contribute more strongly than distant pixels.

Code—Gaussian blur with TensorFlow

```
▶ gaussian_kernel = tf.constant([
    [1., 2., 1.],
    [2., 4., 2.],
    [1., 2., 1.]
]) / 16.0

gaussian_kernel = gaussian_kernel[:, :, tf.newaxis, tf.newaxis]
gaussian_kernel = tf.tile(gaussian_kernel, [1, 1, 3, 1])

gaussian_blur = tf.nn.depthwise_conv2d(
    img_batch,
    gaussian_kernel,
    strides=[1, 1, 1, 1],
    padding="SAME"
)

gaussian_blur = gaussian_blur[0]

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].imshow(img_float)
axes[0].set_title("Original")

axes[1].imshow(tf.clip_by_value(gaussian_blur, 0, 1))
axes[1].set_title("Gaussian Blur")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Code—Result

...

Original



Gaussian Blur



The original Gaussian kernel has shape 3×3 , but `tf.nn.depthwise_conv2d` expects a four-dimensional filter with the form filter height $\times$ filter width $\times$ input channels $\times$ channel multiplier. Therefore, `gaussian_kernel[:, :, tf.newaxis, tf.newaxis]` adds two new dimensions of size 1, changing the shape from 3×3 to $3 \times 3 \times 1 \times 1$. The first new dimension represents the input-channel dimension, while the second represents the channel multiplier. A channel multiplier of 1 means that each input channel produces one corresponding output channel.

The command `tf.tile(gaussian_kernel, [1, 1, 3, 1])` then repeats the tensor along selected dimensions. The first 1 means the kernel is not repeated vertically, the second 1 means it is not repeated horizontally, the 3 repeats the same kernel for the red, green, and blue channels, and the final 1 keeps one output channel for each input channel. Therefore, the kernel shape changes from $3 \times 3 \times 1 \times 1$ to $3 \times 3 \times 3 \times 1$, allowing the Gaussian filter to be applied independently to the R, G, and B channels.

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- **Sharpening**
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Image Sharpening

Sharpening increases local differences between neighboring pixels.

This can make:

- edges more visible,
- object boundaries clearer,
- fine details appear stronger.

A common sharpening kernel is:

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Trade-off

Sharpening can also amplify noise.

Code—sharpening

```
▶ sharpen_kernel = tf.constant([
    [ 0., -1.,  0.],
    [-1.,  5., -1.],
    [ 0., -1.,  0.]
])

sharpen_kernel = sharpen_kernel[:, :, tf.newaxis, tf.newaxis]
sharpen_kernel = tf.tile(sharpen_kernel, [1, 1, 3, 1])

sharpened = tf.nn.depthwise_conv2d(
    img_batch,
    sharpen_kernel,
    strides=[1, 1, 1, 1],
    padding="SAME"
)

sharpened = tf.clip_by_value(sharpened[0], 0.0, 1.0)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

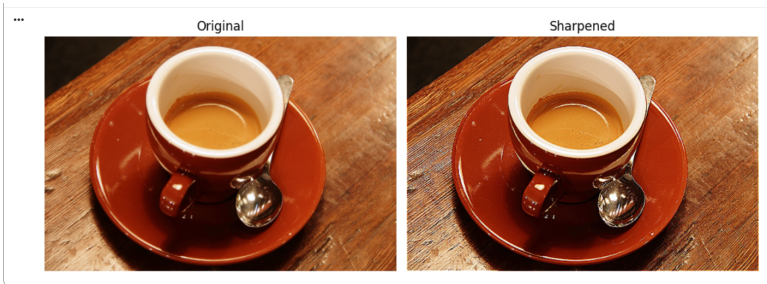
axes[0].imshow(img_float)
axes[0].set_title("Original")

axes[1].imshow(sharpened)
axes[1].set_title("Sharpened")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Code—Result



Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

What Is an Edge?

An **edge** is a location where image intensity changes significantly.

Edges often correspond to:

- object boundaries,
- changes in texture,
- changes in brightness,
- changes in color.

Conceptually:

Small change $\rightarrow$ weak edge

Large change $\rightarrow$ strong edge

Why Are Edges Important?

Object shape and structure are strongly related to where edges occur.

Image Gradients

A gradient measures how quickly image intensity changes.

We often measure changes in two directions:

G_x = horizontal intensity change

G_y = vertical intensity change

The overall gradient strength can be written as:

$$G = \sqrt{G_x^2 + G_y^2}$$

Interpretation

Large gradient values usually indicate strong edges.

How Image Gradients Are Calculated

An image gradient measures how quickly pixel intensity changes. The horizontal gradient can be approximated by comparing pixels to the left and right, while the vertical gradient compares pixels above and below:

$$G_x \approx I(x+1) - I(x-1), \quad G_y \approx I(y+1) - I(y-1)$$

Here, $I(x, y)$ represents the intensity value of the pixel at position (x, y) in the image. For a grayscale image, intensity is simply the pixel value. For an RGB image, a common intensity approximation is $I = 0.299R + 0.587G + 0.114B$, where R , G , and B are the red, green, and blue channel values.

In practice, filters such as the Sobel operator use small kernels that slide across the image and calculate these changes:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The overall gradient strength is then calculated as:

$$G = \sqrt{G_x^2 + G_y^2}$$

Sobel Edge Detection

The **Sobel operator** estimates image gradients using small kernels.

Horizontal-gradient kernel:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical-gradient kernel:

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The Sobel filter convolves this kernel across the image; if neighboring pixel intensities are similar, the positive and negative Sobel weights mostly cancel each other, producing a small gradient value, while a large change in intensity produces a large gradient value that usually indicates an edge.

TensorFlow

TensorFlow provides Sobel edge detection directly with `tf.image.sobel_edges()`.

Code—TensorFlow Sobel edges

```
# tf.image.sobel_edges expects a batch
sobel = tf.image.sobel_edges(img_batch)

print("Sobel shape:", sobel.shape)
```

Sobel shape: (1, 400, 600, 3, 2)

```
# For an input shaped: (1, H, W, 3)
# the output has an extra dimension containing the two gradient directions.

# A clear visualization is easier with grayscale first:
```

Code—Continued

```
▶ gray = tf.image.rgb_to_grayscale(img_float)

gray_batch = tf.expand_dims(gray, axis=0)

sobel = tf.image.sobel_edges(gray_batch)

# Last dimension contains dy and dx
# [batch, height, width, channels, direction] : [batch, height, width, channels, 0 for dy and 1 for dx]
dy = sobel[0, :, :, 0, 0]
dx = sobel[0, :, :, 0, 1]

magnitude = tf.sqrt(dx**2 + dy**2)

fig, axes = plt.subplots(1, 4, figsize=(14, 4))

axes[0].imshow(tf.squeeze(gray), cmap="gray")
axes[0].set_title("Grayscale")

axes[1].imshow(dx, cmap="gray")
axes[1].set_title("Gradient $G_x$")

axes[2].imshow(dy, cmap="gray")
axes[2].set_title("Gradient $G_y$")

axes[3].imshow(magnitude, cmap="gray")
axes[3].set_title("Gradient Magnitude")

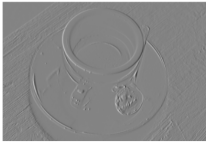
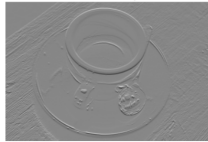
for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

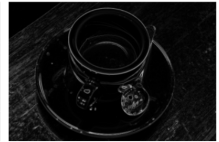
Code—Result

...

Grayscale

Gradient G_x Gradient G_y 

Gradient Magnitude



Note: For Matplotlib, RGB images and grayscale arrays are handled differently: floating-point RGB images are expected to already be in the range $[0, 1]$ and values outside that range are clipped, while 2D grayscale arrays are usually automatically scaled from their minimum and maximum values to the colormap range before display.

Why Blur Before Edge Detection?

Noise also creates rapid pixel changes.

Therefore, an edge detector may mistakenly detect noise as edges.

A common processing pipeline is:

Image $\rightarrow$ Blur $\rightarrow$ Gradient $\rightarrow$ Edges

Intuition

Blurring removes small variations first, allowing the edge detector to focus more strongly on meaningful image structure.

Code— compare Sobel before and after blur

```
▶ gray = tf.image.rgb_to_grayscale(img_float)
  gray_batch = tf.expand_dims(gray, axis=0)

# Original Sobel
sobel_original = tf.image.sobel_edges(gray_batch)
dx_original = sobel_original[0, :, :, 0, 1]
dy_original = sobel_original[0, :, :, 0, 0]
edges_original = tf.sqrt(dx_original**2 + dy_original**2)

# Blur grayscale image
blurred_gray = tf.nn.avg_pool2d(
    gray_batch,
    ksize=5,
    strides=1,
    padding="SAME")

# Sobel after blur
sobel_blurred = tf.image.sobel_edges(blurred_gray)
dx_blurred = sobel_blurred[0, :, :, 0, 1]
dy_blurred = sobel_blurred[0, :, :, 0, 0]
edges_blurred = tf.sqrt(dx_blurred**2 + dy_blurred**2)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

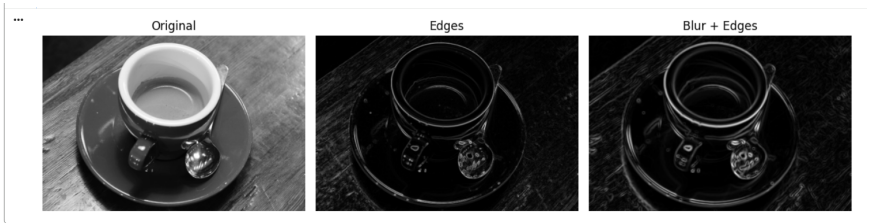
axes[0].imshow(tf.squeeze(gray), cmap="gray")
axes[0].set_title("Original")

axes[1].imshow(edges_original, cmap="gray")
axes[1].set_title("Edges")

axes[2].imshow(edges_blurred, cmap="gray")
axes[2].set_title("Blur + Edges")

for ax in axes:
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Code—Result



Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- **Thresholding and Masks**
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

What Is Thresholding?

Thresholding converts an image into a binary image.

Each pixel is compared with a threshold value T :

$$I_{\text{binary}}(x, y) = \begin{cases} 1, & I(x, y) > T \\ 0, & I(x, y) \leq T \end{cases}$$

Key Idea

Thresholding separates pixels into two groups, such as foreground and background.

Code

```
▶ gray = tf.image.rgb_to_grayscale(img_float)
  gray_2d = tf.squeeze(gray)

  threshold = 0.5
  mask = gray_2d > threshold

  fig, axes = plt.subplots(1, 2, figsize=(9, 4))

  axes[0].imshow(gray_2d, cmap="gray")
  axes[0].set_title("Grayscale")

  axes[1].imshow(mask, cmap="gray")
  axes[1].set_title("Binary Mask")

  for ax in axes:
      ax.axis("off")

  plt.tight_layout()
  plt.show()
```

...

Grayscale



Binary Mask



What Is a Mask?

A **mask** is an image-like array that selects specific pixels.

Usually, a mask contains only two values:

$$0 \quad \text{or} \quad 1$$

A mask can be used to keep some pixels and remove others.

$$\text{Output} = \text{Image} \times \text{Mask}$$

Connection to Segmentation

Image segmentation can be understood as predicting a mask for an object or region.

Code

```
mask_float = tf.cast(mask, tf.float32)
mask_3ch = tf.repeat(mask_float[:, :, tf.newaxis], repeats=3, axis=2)

masked_image = img_float * mask_3ch

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

axes[0].imshow(img_float)
axes[0].set_title("Original")

axes[1].imshow(mask, cmap="gray")
axes[1].set_title("Mask")

axes[2].imshow(masked_image)
axes[2].set_title("Image x Mask")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Original



Mask



Image x Mask



Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- **Morphological Operations**
- Connected Components and Object Counting
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Morphological Operations

Morphological operations modify binary masks using local neighborhoods.

They are commonly used to:

- remove small noisy regions,
- fill small holes,
- expand object regions,
- shrink object regions.

Common Operations

The two basic morphological operations are **erosion** and **dilation**.

Code

```
mask_batch = tf.expand_dims(mask_float, axis=0) # Add a new dimension of size 1 at the beginning for the batch
mask_batch = tf.expand_dims(mask_batch, axis=-1) # Add a new dimension of size 1 at the end for the channel
# [batch, height, width, channels]

dilated = tf.nn.max_pool2d(
    mask_batch,
    ksize=5,
    strides=1,
    padding="SAME"
)

# This uses the identity min(x) = -max(-x), allowing max pooling to perform erosion by selecting the minimum value in each neighborhood.
eroded = -tf.nn.max_pool2d( # Negate the max-pooled result so it becomes the minimum of the original values
    -mask_batch, # Negate the mask so the smallest original values become the largest negative values
    ksize=5,
    strides=1,
    padding="SAME"
)

dilated = tf.squeeze(dilated)
eroded = tf.squeeze(eroded)
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

axes[0].imshow(mask, cmap="gray")
axes[0].set_title("Original Mask")

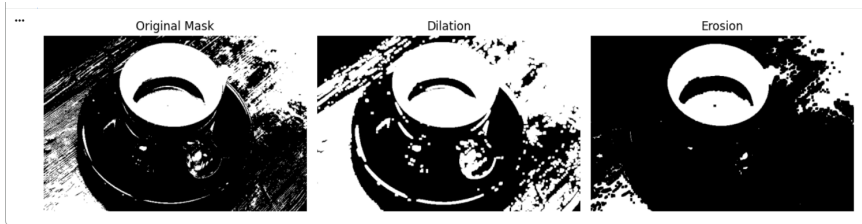
axes[1].imshow(dilated, cmap="gray")
axes[1].set_title("Dilation")

axes[2].imshow(eroded, cmap="gray")
axes[2].set_title("Erosion")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Code—Result



Erosion and Dilation

Dilation expands bright or foreground regions.

Object regions become larger

Erosion shrinks bright or foreground regions.

Object regions become smaller

Intuition

Dilation can fill gaps, while erosion can remove small noisy regions.

How Erosion and Dilation Are Calculated

Both operations use a small neighborhood called a **structuring element** that slides across the image.

For grayscale images, dilation takes the maximum value inside the neighborhood:

$$\text{Dilation: } I_{\text{new}}(x, y) = \max_{(i,j) \in B} I(x + i, y + j)$$

Erosion takes the minimum value inside the neighborhood:

$$\text{Erosion: } I_{\text{new}}(x, y) = \min_{(i,j) \in B} I(x + i, y + j)$$

where B represents the structuring element or local neighborhood.

Interpretation

Dilation replaces each pixel with the brightest value in its neighborhood, causing bright regions to expand, while erosion replaces each pixel with the darkest value, causing bright regions to shrink.

From Classical Processing to CNNs

Classical image processing uses manually designed operations:

blur kernels, edge kernels, thresholds, masks

CNNs also use local image neighborhoods, but instead of manually designing filters, the network learns useful filters from data.

Transition

Classical image processing helps us understand the intuition behind convolutional neural networks.

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- **Connected Components and Object Counting**
- Histograms and Contrast
- Color Spaces
- Geometric Transformations

Connected Components

After creating a binary mask, we may want to identify separate objects.

A **connected component** is a group of neighboring foreground pixels.

Connected components can help us:

- count objects,
- estimate object sizes,
- remove small noisy regions,
- find approximate object locations.

Important

TensorFlow is mainly for tensor operations and deep learning pipelines. For connected components, libraries such as OpenCV or scikit-image are often more convenient.

Code

```
import numpy as np
from skimage import measure

mask_np = mask.numpy().astype(np.uint8)

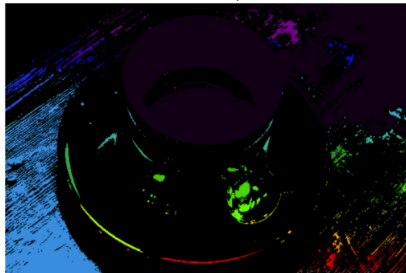
labels = measure.label(mask_np)
num_objects = labels.max()

print("Number of connected components:", num_objects)

plt.imshow(labels, cmap="nipy_spectral")
plt.axis("off")
plt.title("Connected Components")
plt.show()
```

... Number of connected components: 821

Connected Components



Connected Component Labeling

`measure.label()` finds connected foreground regions in a binary mask and assigns a unique integer label to each separate object, while the background remains 0.

0	0	1	1	0	0		0	0	1	1	0	0
0	0	1	1	0	1		0	0	1	1	0	2
0	0	0	0	0	1	$\Rightarrow$	0	0	0	0	0	2
0	1	1	0	0	0		0	3	3	0	0	0

Here, label 1 represents the first connected object, label 2 the second, and label 3 the third. Therefore, `labels.max()` returns 3, which gives the total number of connected objects.

Object Counting from a Mask

A simple object-counting pipeline can look like this:

Image → Grayscale → Threshold → Mask → Connected Components → Count

Limitation

This works well only when objects are clearly separated from the background. For complex images, we usually need more advanced models.

Code

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

axes[0].imshow(img_float)
axes[0].set_title("Original")

axes[1].imshow(mask, cmap="gray")
axes[1].set_title("Binary Mask")

axes[2].imshow(labels, cmap="nipy_spectral")
axes[2].set_title(f"Components: {num_objects}")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

...

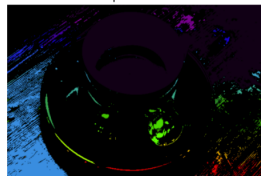
Original



Binary Mask



Components: 821



From Manual Features to Learned Features

Classical image processing often uses manually designed steps:

blur $\rightarrow$ edges $\rightarrow$ threshold $\rightarrow$ mask $\rightarrow$ objects

Deep learning changes the approach:

image $\rightarrow$ neural network $\rightarrow$ learned features $\rightarrow$ prediction

Key Transition

Instead of manually designing every visual feature, CNNs learn useful filters directly from data.

What Should You Remember?

Classical image processing teaches us that images can be changed, analyzed, and understood through numerical operations.

Important ideas from this section:

- pixels are numbers,
- filters use local neighborhoods,
- kernels transform image structure,
- edges represent rapid intensity changes,
- masks select meaningful regions,
- many vision tasks begin with preprocessing.

Next Step

Convolutional Neural Networks use the same local-neighborhood idea, but learn the filters automatically.

Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- **Histograms and Contrast**
- Color Spaces
- Geometric Transformations

Image Histograms

An image histogram shows how often different pixel values occur.

For an 8-bit grayscale image, intensity values range from

0 to 255.

The histogram tells us how many pixels have each intensity.

Why Is It Useful?

Histograms help us understand brightness, contrast, and the overall distribution of image intensities.

Code

```
import matplotlib.pyplot as plt
import tensorflow as tf

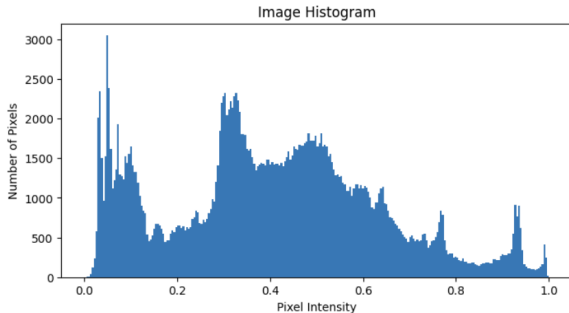
gray = tf.image.rgb_to_grayscale(img_float)
gray_2d = tf.squeeze(gray)

plt.figure(figsize=(8, 4))
plt.hist(gray_2d.numpy().ravel(), bins=256)

plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")
plt.title("Image Histogram")

plt.show()
```

...



Histogram Equalization

Histogram equalization redistributes image intensities to improve contrast.

It can make details more visible when pixel values occupy only a narrow range.

Idea

Spread frequently used intensity values across a wider range.

Important

Histogram equalization can improve contrast, but it may also amplify noise.

In Details

Histogram equalization changes the pixel intensity values so that heavily clustered brightness levels are spread across a wider range, making differences between dark and bright regions more noticeable and therefore increasing image contrast.

Histogram equalization remaps each original intensity value using the cumulative distribution function (CDF), so frequently used intensity levels are spread more widely across the available range:

$$I_{\text{new}} = (L - 1) \text{CDF}(I_{\text{old}})$$

where L is the number of possible intensity levels, such as 256 for an 8-bit image, and $\text{CDF}(I_{\text{old}})$ is the cumulative proportion of pixels with intensity less than or equal to the original pixel value.

Example

Suppose a pixel has intensity r , and:

$$CDF(r) = 0.75$$

For an 8-bit image:

$$L = 256$$

so:

$$s = (L - 1) \times CDF(r)$$

$$s = 255 \times 0.75 \approx 191$$

This means that 75% of all pixels have intensity less than or equal to r , so the original intensity r is mapped to approximately 191, which is about 75% of the available intensity range $[0, 255]$.

Code

```
▶ #TensorFlow does not provide a simple dedicated histogram-equalization function.  
  
import cv2  
  
gray_uint8 = tf.cast(gray_2d * 255, tf.uint8).numpy()  
  
equalized = cv2.equalizeHist(gray_uint8)  
  
fig, axes = plt.subplots(1, 2, figsize=(9, 4))  
  
axes[0].imshow(gray_uint8, cmap="gray")  
axes[0].set_title("Original")  
  
axes[1].imshow(equalized, cmap="gray")  
axes[1].set_title("Histogram Equalized")  
  
for ax in axes:  
    ax.axis("off")  
  
plt.tight_layout()  
plt.show()
```

Code—Result

...

Original



Histogram Equalized



Current Subsection

1 Classical Image Processing

- Why Classical Image Processing?
- Brightness and Contrast
- Noise
- Filtering and Blurring
- Convolution and Kernels
- Gaussian Blur
- Sharpening
- Edges and Gradients
- Thresholding and Masks
- Morphological Operations
- Connected Components and Object Counting
- Histograms and Contrast
- **Color Spaces**
- Geometric Transformations

Beyond RGB: Color Spaces

RGB is not the only way to represent color.

Common color spaces include:

- **RGB**: Red, Green, Blue
- **HSV**: Hue, Saturation, Value
- **YUV**: brightness and color components
- **Grayscale**: one intensity channel

Why Use Different Color Spaces?

Some visual tasks become easier when brightness and color information are represented separately.

Code

```
▶ hsv = tf.image.rgb_to_hsv(img_float)

print("RGB shape:", img_float.shape)
print("HSV shape:", hsv.shape)

fig, axes = plt.subplots(1, 2, figsize=(9, 4))

axes[0].imshow(img_float)
axes[0].set_title("RGB")

axes[1].imshow(hsv)
axes[1].set_title("HSV Representation")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

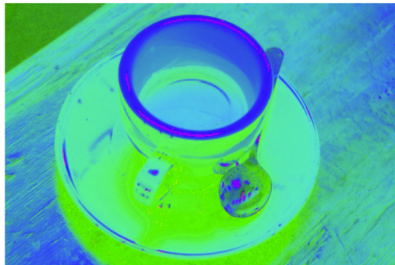
Code—Result

```
... RGB shape: (400, 600, 3)  
    HSV shape: (400, 600, 3)
```

RGB



HSV Representation



Current Subsection

- 1 **Classical Image Processing**
 - Why Classical Image Processing?
 - Brightness and Contrast
 - Noise
 - Filtering and Blurring
 - Convolution and Kernels
 - Gaussian Blur
 - Sharpening
 - Edges and Gradients
 - Thresholding and Masks
 - Morphological Operations
 - Connected Components and Object Counting
 - Histograms and Contrast
 - Color Spaces
 - **Geometric Transformations**

Geometric Image Transformations

Geometric transformations change the spatial arrangement of pixels.

- resize,
- crop,
- flip,
- rotate,
- translate.

These operations are important for both image processing and data augmentation.

Deep Learning Connection

Changing image geometry helps models become less sensitive to exact object position or orientation.

Interpolation During Resizing

When an image is resized, new pixel values must often be estimated. This process is called **interpolation**.

Common methods include:

- nearest-neighbor,
- bilinear interpolation,
- bicubic interpolation.

Key Idea

Interpolation determines how new pixel values are created when the image dimensions change.

How Interpolation Values Are Calculated

Nearest-Neighbor Interpolation: The new pixel takes the value of the closest original pixel:

$$I_{\text{new}}(x, y) = I(\text{nearest original pixel})$$

Bilinear Interpolation (Default): The new location lies between four surrounding pixels. Let α and β represent its relative horizontal and vertical positions within the pixel grid.

$$w_{00} = (1 - \alpha)(1 - \beta), \quad w_{10} = \alpha(1 - \beta), \quad w_{01} = (1 - \alpha)\beta, \quad w_{11} = \alpha\beta$$

where

$$w_{00} + w_{10} + w_{01} + w_{11} = 1$$

The interpolated value is:

$$I_{\text{new}} = w_{00} I_{00} + w_{10} I_{10} + w_{01} I_{01} + w_{11} I_{11}$$

Pixels closer to the new location receive larger weights.

Bicubic Interpolation: Bicubic interpolation uses a 4×4 neighborhood of 16 pixels:

$$I_{\text{new}} = \sum_{i=-1}^2 \sum_{j=-1}^2 I_{ij} w_x(i) w_y(j)$$

The weights w_x and w_y are determined by cubic functions of the horizontal and vertical distances from the new pixel location.

Bilinear Interpolation Example

Each pixel location in the resized image is mapped back to a corresponding location in the original image. This mapped location is often fractional. Suppose the original image is 100×100 pixels and is resized to 200×200 pixels. Consider the pixel at $(x_{\text{new}}, y_{\text{new}}) = (21, 41)$ in the resized image. Its corresponding location in the original image is

$$x_{\text{old}} = x_{\text{new}} \frac{W_{\text{old}}}{W_{\text{new}}} = 21 \frac{100}{200} = 10.50,$$

and

$$y_{\text{old}} = y_{\text{new}} \frac{H_{\text{old}}}{H_{\text{new}}} = 41 \frac{100}{200} = 20.50.$$

Thus, the mapped original-image location is $(x, y) = (10.50, 20.50)$.

Since $x = 10.50$ lies between columns 10 and 11, and $y = 20.50$ lies between rows 20 and 21, the four surrounding pixels are

$$(10, 20), \quad (11, 20), \quad (10, 21), \quad (11, 21).$$

The horizontal fraction is

$$\alpha = \frac{x - x_0}{x_1 - x_0} = \frac{10.50 - 10}{11 - 10} = 0.50.$$

The vertical fraction is

$$\beta = \frac{y - y_0}{y_1 - y_0} = \frac{20.50 - 20}{21 - 20} = 0.50.$$

The four bilinear interpolation weights are

$$w_{00} = (1 - \alpha)(1 - \beta) = (0.50)(0.50) = 0.250,$$

$$w_{10} = \alpha(1 - \beta) = (0.50)(0.50) = 0.250,$$

$$w_{01} = (1 - \alpha)\beta = (0.50)(0.50) = 0.250,$$

$$w_{11} = \alpha\beta = (0.50)(0.50) = 0.250.$$

These weights sum to $0.250 + 0.250 + 0.250 + 0.250 = 1$.

Therefore, the interpolated pixel value is $I_{\text{new}} = 0.250I_{00} + 0.250I_{10} + 0.250I_{01} + 0.250I_{11}$.

The weights are equal because $(10.50, 20.50)$ is equally distant from all four surrounding pixels.

Code

```
▶ nearest = tf.image.resize(  
    img_float,  
    (100, 100),  
    method="nearest")  
  
bilinear = tf.image.resize(  
    img_float,  
    (100, 100),  
    method="bilinear")  
  
bicubic = tf.image.resize(  
    img_float,  
    (100, 100),  
    method="bicubic")  
  
fig, axes = plt.subplots(1, 3, figsize=(12, 4))  
  
axes[0].imshow(nearest)  
axes[0].set_title("Nearest")  
  
axes[1].imshow(bilinear)  
axes[1].set_title("Bilinear")  
  
axes[2].imshow(tf.clip_by_value(bicubic, 0, 1))  
axes[2].set_title("Bicubic")  
  
for ax in axes:  
    ax.axis("off")  
  
plt.tight_layout()  
plt.show()
```

Code—Result

...

Nearest



Bilinear



Bicubic



We Are Ready for Convolutional Neural Networks

So far, we manually selected operations such as:

blur sharpen edges threshold masks

But which filters are best for recognizing a cat, car, or person?

The CNN Idea

Instead of manually designing filters, let the neural network **learn the filters from data**.

Classical Filters $\longrightarrow$ **Learned Filters** $\longrightarrow$ **CNNs**

Congratulations!

Now you know about classical image processing!

